Harness Engineering

横纵分析法深度研究报告

研究时间：2026年4月 | 所属领域：AI Engineering / Agent Runtime / 大模型应用基础设施 | 研究对象类型：技术范式 / 工程方法论

作者: 派蒙

> 研究时间：2026年4月 I 所属领域：AI Engineering / Agent Runtime / 大模型应用基础设施 I 研究对象类型：技术范式 / 工程方法论

一、一句话定义

Harness Engineering 是围绕大模型外层运行时所做的一整套工程设计：它不再只关注“怎么提示模型”，而是关注“怎么给模型配置上下文、工具、记忆、状态机、验证闭环和恢复机制”，让模型能够在长任务、多步骤、低容错的真实环境里稳定工作。说得更直白一点，它是在给大模型搭一层“操作系统”。

二、纵向分析：Harness Engineering 是怎么长出来的?

2.1 前史：Prompt Engineering 为什么会越来越不够用?

大模型应用的最早阶段，行业几乎是被 Prompt Engineering 统治的。

那时候大家的核心问题很简单：**怎么跟模型说话，模型才会听懂?**

于是方法一波接一波地冒出来：
- 角色设定
- few-shot 示例
- chain-of-thought
- system prompt 分层
- 分步指令
- 输出格式约束

这些东西不是没用，恰恰相反，它们非常有用。问题在于，它们主要适用于一种场景：**单轮或短链路任务**。

也就是说，当模型只是帮你写一段文案、总结一篇文章、回答一个问题时，提示词几乎就是全部。

但一旦任务开始变复杂——比如：
- 让模型跨几十个文件改一个功能
- 让它连续工作两小时写研究报告
- 让它一边检索、一边浏览网页、一边写代码、一边验证结果
- 让它在企业知识库、项目系统、工单系统、办公文档之间来回穿梭

你就会发现，提示词像一张口头承诺。它能告诉模型“你应该怎么做”，但它没法保证模型真的一直照做。

Prompt Engineering 最大的问题，不是提示词写得不够好，而是它天然缺乏以下几种能力：

1. **状态管理能力**：它不知道任务做到哪一步了
2. **记忆管理能力**：它不知道哪些信息该留、哪些该丢
3. **恢复能力**：一旦中断，很难从原地继续
4. **验证能力**：模型会说自己做完了，但不一定真做完了
5. **约束能力**：提示词没法物理上阻止模型乱来

这就是为什么 Prompt Engineering 后来慢慢不再像 2023 年那样是“万法之源”。不是它失效了，而是它被降维成了更大系统里的一部分。

2.2 RAG、Tool Use、Agent：行业开始从“说服模型”转向“组织模型”

真正的变化，大概发生在三条支线同时成熟之后。

第一条支线是 **RAG**。它让行业第一次真正认真地面对一个问题：模型不是全知的，那外部知识该怎么接进去？

RAG 的贡献不是“检索”本身，而是让大家接受了一种新思路：**模型外部应该有一个系统来供给信息。**这一步很关键，因为它意味着“模型不再是封闭大脑，而是一个需要外部喂养的推理内核”。

第二条支线是 **Tool Use / Function Calling**。OpenAI 在 2023 年把 function calling 做成正式能力之后，模型第一次不只是“说”，而是能“调工具”。Anthropic 后面跟进 tool use、再到 computer use，彻底把这一方向推到了新阶段。

这件事的意义很大。因为一旦模型能调工具，它就不再是聊天机器人，而开始接近一个“执行体”。执行体意味着什么？意味着你必须关心：
- 它可以调哪些工具
- 什么情况下调
- 调完结果怎么回填
- 失败了怎么重试
- 工具太多会不会乱用

第三条支线是 **Agent 化**。AutoGPT、LangChain、Devin、Claude Code、Cursor 这一批产品和框架，实际上共同把行业从“问一个问题得到一个答案”推进到了“给它一个目标，它自己去干”。

而一旦进入 agent 时代，问题立刻变得工程化。

Prompt 不再只是提示，而变成工作流的一小部分。真正的难点开始变成：

> **这个模型在接下来的一小时里，该怎么看信息、怎么记住状态、怎么调用工具、怎么避免跑偏、怎么在失败后回来继续。**

这时候，Harness Engineering 就开始从隐性经验变成显性概念了。

2.3 2024：Context Engineering 开始冒头，行业第一次意识到“给什么信息”比“怎么写提示词”更重要

如果说 2023 年还是 Agent 化的狂热，那么 2024 年开始，行业逐渐冷静下来，开始面对一堆非常真实的失败。

这些失败不是模型不会回答问题，而是模型在长任务里表现得像一个聪明但极不稳定的实习生：
- 你让它连续干半小时，它开始忘事
- 给它太多上下文，它开始乱
- 提示词写得再详细，它也会慢慢偏离原始目标
- 它会为了完成任务疯狂调工具，结果把 token 烧光
- 它经常说“我已经完成了”，实际上只是看起来完成了

于是一个新的概念开始流行：**Context Engineering**。

这比 Prompt Engineering 更进一步。Prompt Engineering 关心的是“怎么说”，Context Engineering 关心的是“到底让模型看到什么”。

这其中最关键的认知转变是：

> **上下文不是越多越好，而是越对越好。**

大模型不是读书越多越聪明的学生，而更像一个在当前工作记忆里运转的系统。你塞太多无关内容，它不是变强，而是变钝。

所以从 2024 年开始，大量工程实践开始围绕以下问题展开：
- 上下文怎么裁剪
- 历史对话怎么总结
- 什么内容放 system prompt，什么内容延迟注入
- 工具返回结果怎么压缩
- 检索信息怎么防噪声
- 超长任务什么时候该重置上下文

这其实已经离“写提示词”很远了。你会发现它越来越像系统设计。

2.4 Anthropic 为什么会成为这一方向最重要的推动者之一？

如果只从公开资料看，Anthropic 很可能不是最早发明“Harness Engineering”这个词的人，但几乎可以说，它是**最早把这一整套问题系统化地讲清楚、做成产品、再写成方法论**的公司之一。

原因有三层。

第一层：Claude 这一路非常强调“强模型 + 长上下文 + 真实任务”

Anthropic 做 Claude 时，一条很鲜明的路线就是：
- 做更强的长上下文
- 做更可靠的 tool use
- 做 computer use
- 做 Claude Code 这种真正落地的 agent 产品

它不是停留在 API 能力，而是持续把模型推进到“真实世界执行”里去。你一旦把模型推进到真实任务中，Harness 问题就无法回避。

第二层：Anthropic 很愿意写“失败是怎么发生的”

这一点特别重要。很多公司会展示结果，但不愿意系统讲失败模式。Anthropic 相对不同，它公开讨论了大量 agent 的失败现象：
- 上下文焦虑
- 提前交卷
- 虚标完成
- 失忆式中断
- 自我验证偏差

这类工程观察的价值很高，因为它让行业第一次能把 agent 失败理解成“结构性问题”，而不是“模型偶尔抽风”。

第三层：Claude Code 把理念做成了产品化范式

Claude Code 很重要，不只是因为它是 coding agent，而是因为它把很多 harness 思路产品化了：
- `CLAUDE.md` 作为项目级规则和上下文入口
- scratchpad / task docs 作为外部记忆
- planner / evaluator 风格的任务分解与校验
- 对长任务过程的结构化管理

你可以把它理解成一种公开的范式示范：**模型不是裸奔工作的，它是在一个精心布置的工作环境里工作。**

所以如果要回答“Claude 是否是 Harness Engineering 的开创者”，我会更谨慎地说：

> **Anthropic / Claude 至少是这一方向最重要的理念推进者、问题提出者和产品化示范者之一。**

它不一定独占发明权，但它显然在塑造行业共识。

2.5 2025-2026：Harness Engineering 从隐性经验升级为显性方法论

到了 2025-2026，行业开始出现一个明显变化：

大家不再只是交流 prompt，不再只是晒 agent demo，而是开始正经讨论：
- 任务状态怎么持久化
- 什么时候做 context reset
- 工具调用应该多自由、多受限
- agent 之间怎么分工
- 验证器要不要和生成器分离
- planner、worker、judge 这种结构是不是必要

这一阶段，一个特别重要的共识逐渐形成：

> **Agent = Model + Harness**

意思是说，一个 agent 的实际能力，不等于模型本身的能力，而等于模型能力乘上它外层工程的质量。

这个结论非常像早年分布式系统或者数据库行业的经验：底层引擎很重要，但真正决定系统是否能用于生产环境的，是调度、恢复、日志、状态、约束、观测、容错。

所以“Harness Engineering”其实是大模型行业的一次成熟化信号。它意味着行业开始从“觉得模型很神奇”进化到“承认模型是一种需要工程驯化的能力内核”。

三、横向分析：不同路线，究竟在怎么给模型搭“外层系统”？

3.1 Anthropic / Claude：强模型优先，Harness 是必要但克制的补偿层

Anthropic 这一派的特点很鲜明：**先相信强模型，在足够好的上下文里能自己做很多事；Harness 主要用来解决它做不到或做不稳的部分。**

这条路线的关键词包括：
- 长上下文
- prompt caching
- tool use
- computer use
- Claude Code
- context reset

它更像“模型能力放大型”。

优点是：
1. 如果模型足够强，系统复杂度可以压低
2. 大量任务不必过度显式编排

3. 用户体验更自然，像在和一个真正会工作的 agent 协作

局限也很明显：

1. 有些运行时逻辑仍偏黑盒
2. 开发者对内部机制控制度不如显式框架高
3. 一旦模型本身在某类任务上失误，产品方需要非常聪明地补工程

Anthropic 这一路其实代表了一种很“模型中心主义”的工程哲学：**不要过早假设模型不行，但要为它准备好恢复和纠偏结构。**

3.2 OpenAI：把 Harness 的很多部分做成平台标准件

OpenAI 的路线明显更偏“平台层”。

如果 Anthropic 更像在讲“怎么让模型在长任务里自己工作”，OpenAI 更像在讲“怎么给开发者一套标准接口，让你自己去组织模型工作”。

它的关键词包括：

- function calling / tool calling
- structured outputs
- responses API
- assistants / agents 风格托管能力
- Codex 的工程实验

OpenAI 更像“API / 平台封装型”。

优点：

1. 标准化程度高，开发者接入成本低
2. 能力组件清晰，便于大规模生态扩展
3. 工具协议和结构化输出非常适合作为通用基础层

局限：

1. 它给你的是积木，不一定是成品
2. 对复杂 runtime 的最佳实践，需要开发者自己补
3. 如果没有很强的产品团队，光有 API 仍然未必能做出优秀 agent

OpenAI 的价值更像云厂商——它在搭高速公路，而不是给你直接造最终的物流系统。

3.3 Google / Gemini：超长上下文与生态接入，但 Harness 心智还不够强

Google / Gemini 这一路最突出的，是长上下文和 Workspace / 企业生态整合能力。

这使它天然适合办公场景、文档密集场景、企业协同场景。它的逻辑更像：**如果我能把用户整个工作环境都接进来，模型就能在更丰富的信息环境里直接推理。**

这也是一种模型能力放大型思路。

但它和 Anthropic 的区别是：Anthropic 在 agent failure modes 和 harness 方法论上讲得更清楚，而 Google 公开层面的这部分叙事相对弱。也就是说，它在模型和生态上很强，但在“如何组织 agent runtime”这个开发者心智上，目前没有前两者那么鲜明。

3.4 LangChain / LangGraph：最像“显式编排器”的路线

如果前面几家更多是产品或平台，那么 LangGraph 代表的是另一种完全不同的思路：

> **别相信模型自己会自然把任务走对，直接把流程图、状态图、检查点、节点依赖显式建出来。**

这就是典型的“显式编排框架型”。

它特别适合：
- 企业复杂流程
- 多步骤审批和分支逻辑
- 需要可观测、可追踪、可恢复的工作流
- 多 agent 协作

优势：
1. 可控性强
2. 可观测性强
3. 适合复杂业务逻辑和工程约束

局限：
1. 开发复杂度高
2. 学习曲线陡
3. 更适合工程师，不如成品产品自然

LangGraph 的意义很像 Kubernetes 对容器世界的意义：它不一定是终端用户最爱直接接触的东西，但它会成为大量复杂系统背后的编排层。

3.5 Cursor：为什么很多人觉得它“顺手”，本质上是 Harness 赢了

Cursor 是一个非常好的例子，用来证明“用户最终感受到的 agent 好不好，很大程度不是模型本身决定的”。

Cursor 的核心优势并不只是它能接哪个模型，而是它把 agent 放进了**IDE / workspace 原生环境**里：
- 它知道你正在看哪个文件
- 它知道工作区有哪些文件
- 它可以围绕代码编辑、diff、规则、计划面板去组织交互
- 它不是抽象调用模型，而是在真实开发行为流中嵌入 agent

这使它成为典型的“产品化 agent runtime 型”。

Cursor 强的地方，不是把编排暴露给你看，而是把很多 harness 设计藏进了体验里。所以用户感知到的是："它更懂我现在正在干什么。"

这也是为什么很多时候，用户会觉得 Cursor 比某些理论上模型更强的平台更好用——因为它的 runtime 更贴近真实任务。

3.6 Devin：把自主性推到最远，也把 Harness 的代价暴露得最彻底

Devin 代表另一条路线：**真正的自主执行闭环。**

它不是辅助式 coding assistant，而是试图成为"自己把事情做完"的 AI 工程师。为此，它必须同时拥有：

- planner
- shell
- browser
- 文件系统
- 独立工作环境
- 长任务状态管理

这使它非常接近"产品化 runtime 的极端形态"。

它的优点很清楚：

- 任务闭环完整
- 自主性叙事强
- 对用户来说很像"把任务交给另一个人"

但它的代价也最大：

- 成本高
- 失败恢复复杂
- 错误一旦发生，传播链更长
- 对 harness 的要求远高于辅助式产品

Devin 特别适合说明一个道理：**Agent 越自主，Harness 就越不能出错。**

四、横纵交汇洞察：Harness Engineering 真正改变了什么？

4.1 它把"模型能力"重新拆成了"模型 + 系统"

过去大家总爱问："哪个模型更强？"

Harness Engineering 起来之后，问题变成了：

- 这个模型放在什么上下文结构里？
- 它调用哪些工具？

- 它的状态怎么存?
- 它失败以后怎么恢复?
- 谁来验证它是不是真的完成了?

也就是说，**模型不再是最终产品，而是一个需要被编排的推理引擎。**

这会带来一个很深远的后果：模型厂商和产品厂商之间的边界开始重新划分。模型厂商提供的是认知内核，真正让这颗内核在生产环境中持续工作的，是 runtime / harness 层。

4.2 Harness Engineering 解决的是“真实任务失败”，不是概念营销

最让我觉得这个方向站得住的，不是某家公司提出了一个词，而是它确实在解决一组非常具体、可重复观察的失败：

- 长上下文污染
- prompt drift
- tool spam
- memory 召回噪声
- 中断后无法恢复
- 多文件状态错乱
- 自我验证失真

这些问题的共同点是：

> **它们都不是靠一句更好的提示词就能解决的。**

这就是 Harness Engineering 合理性的根基。它不是替代 Prompt Engineering，而是把 Prompt Engineering 放回了它该在的位置：一个子模块，而不是全部。

4.3 哪些场景最依赖 Harness? 答案几乎已经定了

目前看，最依赖 harness 的五类场景非常清楚：

1. **coding agents**：因为任务长、状态复杂、验证刚性强，是最容易暴露 harness 优劣的场景
2. **research agents**：因为多来源检索、多跳推理、引用追踪都需要系统管理
3. **enterprise copilots**：因为权限、知识、审计、工具接入决定其是否真能落地
4. **workflow automation**：因为状态机和异常处理决定它是不是企业级系统，而不是 demo
5. **browser / computer use agents**：因为现实环境变化大、脆弱点多，没有 runtime 根本不稳

这里面最先成熟、最能产生清晰商业壁垒的，还是 coding agents。因为 coding 场景对“能不能真正把事做完”的要求最刚，最适合检验 harness 的水平。

4.4 长期壁垒到底来自模型，还是 Harness?

这是最值得给一个平衡回答的问题。

我的判断是：

> **短期看，Harness 决定产品体验；长期看，模型能力 + 真实任务数据飞轮共同决定天花板。**

为什么短期是 Harness?
因为同一模型，只要换一个更好的 runtime，体验就可能完全不同。用户感受到的稳定性、恢复能力、工作流连贯性，很多都来自外层系统。

为什么长期又不能只说 Harness?
因为模型能力进步会改变 harness 的复杂度边界。今天必须显式编排的东西，明天也许模型自己就能处理一部分。与此同时，谁拥有更多真实任务轨迹数据，谁就能更快优化自己的编排层和模型协同方式。

所以最终壁垒不是“模型 vs Harness”的二选一，而是：

- 模型能力决定上限
- Harness 设计决定落地质量
- 真实任务轨迹数据决定进化速度

这三者会越来越耦合。

4.5 未来趋势：Context Engineering 会变成新的 Prompt Engineering 吗?

我觉得答案基本是：**会，而且已经在发生。**

但严格说，未来行业重心可能不只是 Context Engineering，而是一个更大的层级：

> Prompt Engineering → Context Engineering → Harness Engineering → Agent Runtime Engineering

也就是说，未来最有价值的能力，不再是会写神提示词，而是会设计：
- 工作区记忆
- 会话状态
- 工具协议
- 验证环
- 中断恢复
- 多 agent 协作
- 企业知识接入

如果说 2023 年大家在学“怎么跟模型说话”，那么 2026 之后，真正的高级能力很可能是：

怎么给模型设计一个能长期工作的环境。

这会让 AI 工程越来越像系统工程，而不是内容创作技巧。

五、结论：Harness Engineering 的意义，不是让模型更聪明，而是让模型真正能上班

研究到最后，我觉得 Harness Engineering 最值得记住的一点，是它特别“去魅”。

它不像 prompt engineering 那样有点神秘感，不像 model scaling 那样有点宏大叙事。它干的事情非常朴素：

- 别让模型忘事
- 别让模型乱来
- 别让模型自我感动
- 别让模型一断就废
- 别让模型在真实工作里看起来聪明、实际没交付

说到底，它做的不是提升智商，而是提升**职业素养**。

一个没有 harness 的模型，很像一个知识很多、表达很强、但没有流程意识、没有状态感、做事不留痕、做错了也不会自查的高智商实习生。

一个有优秀 harness 的模型，才更像真正能加入团队的工作者。

所以 Harness Engineering 真正改变的，不是我们怎么看 prompt，而是我们怎么看 agent。

它告诉我们：

> **大模型不是“会回答问题”就够了。要让它真正创造价值，你得先给它一张桌子、一套流程、一堆工具、一份记事本、一个审稿人，最好还得有个能在它犯错时把它拽回来的系统。**

这就是 Harness。

也是为什么我会把它理解成：

从“写提示词”，走向“给模型搭操作系统”。

六、信息来源

官方与一手资料

- Anthropic 官方 engineering / research 博客（context engineering、harness、computer use、Claude Code相关）
- OpenAI 官方关于 function calling / responses / Codex / harness engineering 的文档与博客
- Google / Gemini 官方文档与 Workspace 集成资料
- LangChain / LangGraph 官方博客与 Terminal Bench 相关文章
- Cursor 官方 engineering / best practices 博客
- Stripe Minions 工程博客

典型参考链接

- https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents
- https://www.anthropic.com/engineering/harness-design-long-running-apps
- https://www.anthropic.com/news/claude-computer-use
- https://openai.com/index/harness-engineering/
- https://blog.langchain.com/the-anatomy-of-an-agent-harness/
- https://www.langchain.com/blog/improving-deep-agents-with-harness-engineering
- https://cursor.com/blog/agent-best-practices
- https://stripe.dev/blog/minions-stripes-one-shot-end-to-end-coding-agents
- https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html
- https://blog.can.ac/2026/02/12/the-harness-problem/

补充分析与社区资料

- JavaGuide 等中文技术综述
- GitHub / Hacker News / 技术博客中关于 context engineering、agent runtime、Claude Code、Cursor、Devin 的讨论

方法论说明：

本报告采用**横纵分析法**（Horizontal-Vertical Analysis）：
- **纵向**：追踪从 Prompt Engineering 到 Context Engineering，再到 Harness Engineering 的范式演进
- **横向**：比较 Anthropic、OpenAI、Gemini、LangGraph、Cursor、Devin 等路线在 runtime harness 上的不同思路
- **交汇**：分析它为什么成为新的工程能力、产业壁垒与未来趋势

报告完成时间： 2026年4月22日

研究对象： Harness Engineering

数据截止： 2026年4月

作者： 派蒙（基于横纵分析法框架）